

CGRA-EAM—Rapid Energy and Area Estimation for Coarse-grained Reconfigurable Architectures

MARK WIJTVLIET, Dresden University of Technology, Germany

HENK CORPORAAL, Eindhoven University of Technology, Netherlands

AKASH KUMAR, Dresden University of Technology, Germany

Reconfigurable architectures are quickly gaining in popularity due to their flexibility and ability to provide high energy efficiency. However, reconfigurable systems allow for a huge design space. Iterative design space exploration (DSE) is often required to achieve good Pareto points with respect to some combination of performance, area, and/or energy. DSE tools depend on information about hardware characteristics in these aspects. These characteristics can be obtained from hardware synthesis and net-list simulation, but this is very time-consuming. Therefore, architecture models are common. This work introduces CGRA-EAM (Coarse-Grained Reconfigurable Architecture - Energy & Area Model), a model for energy and area estimation framework for coarse-grained reconfigurable architectures. The model is evaluated for the Blocks CGRA. The results demonstrate that the mean absolute percentage error is 15.5% and 2.1% for energy and area, respectively, while the model achieves a speedup of close to three orders of magnitude compared to synthesis.

CCS Concepts: • **Computer systems organization** → **Reconfigurable computing**; • **Hardware** → **Modeling and parameter extraction**

Additional Key Words and Phrases: CGRA, reconfigurable architecture, energy efficiency

ACM Reference format:

Mark Wijtvliet, Henk Corporaal, and Akash Kumar. 2021. CGRA-EAM—Rapid Energy and Area Estimation for Coarse-grained Reconfigurable Architectures. *ACM Trans. Reconfigurable Technol. Syst.* 14, 4, Article 19 (September 2021), 28 pages.

<https://doi.org/10.1145/3468874>

1 INTRODUCTION

Many coarse-grained reconfigurable architectures have emerged over time [6, 22]. In part, this can be attributed to increased performance and cost reduction. Post-production updates and rapid development of new algorithms and standards are another reason. Whereas ASICs would require silicon updates and a new production run, devices like **field programmable gate arrays (FPGAs)** can be reconfigured to correct an error or implement new functionality. Reconfigurability comes, of course, at a cost. Taking FPGAs as an example: The reconfigurable interconnect causes longer wires between gates than those in ASICs, reducing the maximum frequency and performance and increasing area and power. The large configuration memory controlling the interconnect and

Authors' addresses: M. Wijtvliet and A. Kumar, Dresden University of Technology, Helmholtzstrasse 18, Dresden, Sachsen, Germany, 01069; emails: {mark.wijtvliet, akash.kumar}@tu-dresden.de; H. Corporaal, Eindhoven University of Technology, Groene Loper 19, Eindhoven, Netherlands, 5612 AZ; email: h.corporaal@tue.nl.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2021 Association for Computing Machinery.

1936-7406/2021/09-ART19 \$15.00

<https://doi.org/10.1145/3468874>

operation of the configurable logic causes high static power dissipation [3, 11], further reducing energy efficiency. Additionally, the logic area of such FPGAs is generally much bigger.

Coarse-grained reconfigurable architectures (CGRAs) allow reconfiguration at a granularity above gate-level, such as functional units or lightweight processors that can be connected via a static or dynamically reconfigurable network. In DSP applications, where it is much more common to use floating-point or fixed-point arithmetic rather than bit-level operations, CGRA-style functional units are more efficient in performance, area, and energy compared to FPGAs.

Although CGRAs can be very efficient, designing efficient configurations for reconfigurable architectures is not trivial and usually takes many design iterations. For each design iteration, a designer requires information on the performance, energy, and area of the configuration to make informed choices. The same holds for automated design tools that synthesize a high-level language description into hardware configurations and instruction schedules. These instruction schedules are generated by a compiler during the translation of an input application to machine code. These schedules describe which instructions are mapped to which processing resources and in which order they will be executed. This can provide performance and hardware utilization information for an energy model. Performance information can also be obtained by simulation of the designed instruction schedule; this will yield an exact cycle count. Most architectures also allow for execution time analysis by automated design tools as long as the execution time is not significantly data-dependent.

Energy and area estimation usually requires at least a complete logic synthesis run. For a reasonably sized design, this takes hours to complete. Although these energy and area estimations are very accurate, taking several hours for each design iteration is far too long when many iterations have to be performed. It is possible to use pre-synthesized macros for building blocks such as function units and switch-boxes. This will improve the synthesis speed significantly. However, this may come at the expense of a less optimized system-level design, as no cross module optimizations are made. Furthermore, it is still required to perform RTL level simulation and power extraction to obtain energy consumption. This still takes a significant amount of time.

Using a model for energy and area estimation will be less accurate but several orders of magnitude faster. Even though a model may be less accurate in terms of absolute energy and area numbers, the way a model scales for small design changes is usually accurate enough for **design space exploration (DSE)**.

Since efficient mapping of an application to CGRAs is non-trivial and can be time-consuming when done manually, it is important to automate these design stages as much as possible for CGRAs to become a success outside of the research environment.

Contributions. This article introduces an energy and area model for CGRAs that enables fast exploration of architecture properties. Although this article uses the Blocks CGRA for evaluation, the methodology of profiling individual building blocks to estimate system-level energy works, in principle, for any CGRA, as long as a synthesizable hardware design is present. Blocks is used for evaluation, as this is an architecture for which a synthesizable design is available.

- A method to extract architectural characteristics of CGRAs, using micro-benchmarks on a post-synthesis net-list.
- A new architectural model to estimate energy and area for CGRAs, allowing for fast design-space exploration.
- An extensive evaluation of the proposed model, and extracted architecture characteristics, with respect to energy and area.

Section 3 introduces how energy and area information is obtained for the Blocks architecture. Section 4 introduces the proposed architectural model. Section 5 provides an evaluation on the accuracy of the model and its execution time. Section 6 concludes this work.

2 RELATED WORK

Most models and DSE frameworks for CGRAs focus on performance, some examples thereof are References [12, 13, 24, 25]. One of the most cited CGRA modeling frameworks is the CGRA-ME framework [2], which provides a tool-flow to obtain performance information from hardware descriptions of arbitrary CGRAs. To do so, an architecture is described in a custom XML-based language called CGRA-ADL. From this language, a software model of the CGRA is constructed that represents the physical logic and interconnect resources of the CGRA. A data-flow representation of a benchmark application is then mapped to this model to obtain performance numbers. The tool-flow allows generation of synthesizable Verilog RTL, which can be used to characterize the function units. This is a requirement for every framework that wishes to integrate the CGRA-EAM energy and area estimation model.

The work presented in Reference [8] uses efficient arrangement of reconfigurable array components and the interconnect to exploit the memory access patterns present in digital signal processing applications. Although a breakdown of area and energy is given for a baseline application as well as several benchmark applications, the proposed DSE tool does not use an area or power model to achieve these results. The power and area reductions are achieved by optimizing the memory access pattern and mapping of the application to the hardware resources. In contrast to this work, there are energy aware synthesis tools [16] that do take into account energy during synthesis. In this case, empirical, relative scaled, power values are used to estimate energy scaling during design exploration. CGRA-EAM, however, uses actual hardware characterization of post-synthesis net-lists to obtain absolute energy consumption. Rather than providing scaling information, CGRA-EAM provides quick but accurate absolute energy and area information for an architecture instance.

Recent work [14] uses micro-benchmarking to train a machine learning model. This model is subsequently used to analyze application performance. A similar approach could be used for energy and area as well. However, if inaccuracies in the final estimations do occur, then it is much harder to determine the cause of these deviations. Therefore, CGRA-EAM uses an analytical method to estimate energy and area. The work presented in Reference [7] also uses an analytical model to estimate power. However, this is done on a much more fine-grained level than the method proposed in this work. Furthermore, data-types are not taken into account, whereas CGRA-EAM does.

Power and path delay estimation has been done for FPGAs [1]. In this work, system power is estimated based on the utilization levels of interconnect and multiplexer resources. The results of this work show a mean error of 21%. However, the overhead of the interconnect is much lower for CGRAs compared to FPGAs. Due to this, the interconnect and multiplexer utilization may not be a good basis for CGRA energy estimation. Rapid design-space exploration for multiple use cases is explored in Reference [10]. In this work a full design space exploration for an FPGA design, including mapping and synthesis, are performed in 45 minutes. A good underlying FPGA model allows such DSE; the goal of CGRA-EAM is to enable a similar methodology for CGRAs as well.

ASIC design tools also perform power estimation. Traditionally, this is done based on a library of characterized standard cells. This information is then directly referenced to estimate the power for an IP Block. Another way of estimating power for IP blocks is to use a power contributor model [4]. In such a model the physical parameters such as switching capacitance, resistances, and

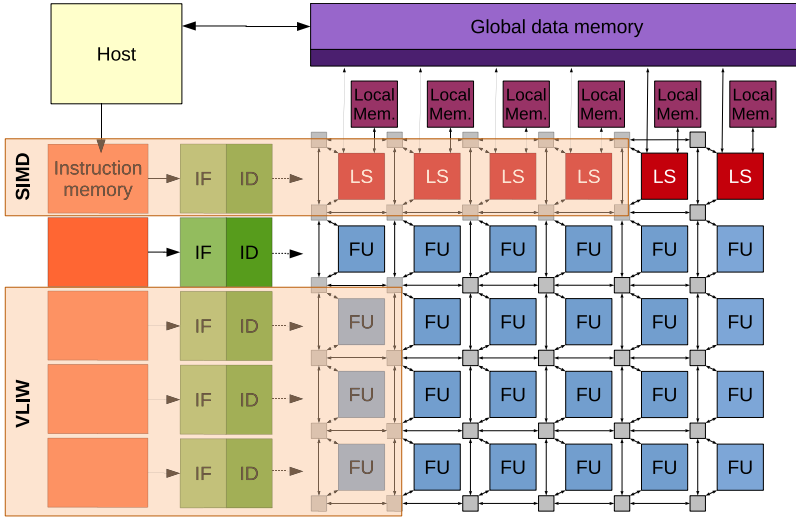


Fig. 1. Both data-level parallel (SIMD) and instruction-level parallel (VLIW) processors can be instantiated onto the Blocks fabric [23].

current sources are stored instead of extracted power numbers. This has the advantage that the model is frequency, process, voltage, and temperature. Another method is to use a hybrid power model [17] that uses transistor-level information at gate level. This helps to increase the power estimation accuracy but does not require electrical simulations. Both of these methods significantly increase power estimation with respect to traditional simulations. These models achieve this by abstracting from the fine-grained building blocks. CGRA-EAM uses a similar technique but at a higher abstraction level. Instead of abstracting from transistors to standard cells, CGRA-EAM abstracts from standard cells to CGRA building blocks, such as function units.

The method presented in this work is targeted at the Blocks reconfigurable architecture, which is used for evaluation of the model. However, the function unit characterization method, and area and energy estimation, can be used for many different CGRAs such as KAHRISMA [9], TRIPS [19], ADRES [15], and RAW [21], as long as RTL implementations of these architectures are available or can be generated.

3 FUNCTION UNIT CHARACTERIZATION

Energy and area models require information about the characteristics of an architecture. These characteristics can be determined by performing micro-benchmarks on individual building blocks of an architecture. This article proposes an energy and area model for the CGRAs.

3.1 The Blocks Reconfigurable Architecture

For characterization and evaluation the Blocks architecture will be used [23]. Blocks is a coarse-grained reconfigurable architecture, consisting of a 2D mesh of function units that can be connected to each other to form a data-path. Unique to Blocks is that data-parallel vector lanes can be instantiated on the reconfigurable fabric. This allows Blocks to control multiple (parallel operating) function units to be controlled by a single instruction decoder, as shown in Figure 1, to create an SIMD processor. Similarly, multiple instruction decoders can be used to construct VLIW processors.

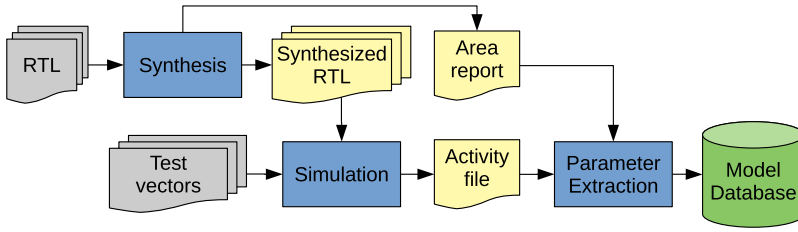


Fig. 2. Tool-flow for architecture parameter extraction.

The function units are heterogeneous and, by default, include six different types: a **load-store unit (LSU)**, an **arithmetic and logic unit (ALU)**, a **multiplier unit (MUL)**, an **accumulate and branch unit (ABU)**, **register files (RF)**, and an **immediate unit (IU)**. The ABU generates the program counter and is used to sequence the instruction decoders.

3.2 Method for Characterizing CGRA Building Blocks

Since CGRAs, including Blocks, usually have a relatively limited number of building blocks (function units, switch-boxes, instruction decoders) it is feasible to individually profile these units as a one-time effort and obtain energy consumption for the operations these building blocks can perform. Based on the properties of the individual building blocks, the total energy and area of the architecture can be estimated using a model. This model cannot simply add metrics to obtain a result. For example: in Blocks the size of the arbiter scales with the number of LSUs, instead of evaluating every arbiter size the area can be interpolated or extrapolated based on known design points. Furthermore, energy is significantly impacted by activity. Therefore, the model performs a first-order approximation on energy based on specified data-types in the instruction schedule. Figure 2 shows the tool-flow to obtain architecture properties. Specifically, parameters for power and area estimation. The hardware description in RTL is provided to a regular synthesis tool that produces a net-list and area report. The synthesized net-list is then simulated for a series of test vectors that results in an activity file. This file is then used to estimate power for individual building blocks and instructions. The results are stored in a database that will be used by the model described later in this article. This section details the characterization process in more detail.

This process is almost fully automated. The only manual step that is required is to specify a configuration for the test-benches used for characterization. In this configuration, the instructions to be characterized, their number of operands, and supported data types are specified. This has to be done once for each function unit type. In the future, this step can be automated, as the data is available within the Blocks framework. However, as this needs to be done only once for a few function units, this was performed manually. The remaining steps are fully automated. For each function unit the configuration file is processed, the hardware description is synthesized, and the results are gathered and parsed for storage in the model database.

Obtaining accurate energy consumption for the individual building blocks requires applying known inputs to the **design under test (DUT)**. This requires a test-bench that, after synthesis of the design under test (DUT), applies a sequence of inputs while an activity trace of all internal nets of the DUT is recorded. Within the Blocks framework, the test-bench for most DUTs could be formed by a slightly modified version of a hard-wired Blocks instance. This instance contains some function units for execution control (a branch unit (ABU), ALU, and immediate unit) as well as a test-bench controller. The function units load an application that controls their operation, and with this the program flow. The test-bench controller is a sequencer that provides inputs to the DUT synchronously with program execution. Figure 3 shows the structure of the design used for

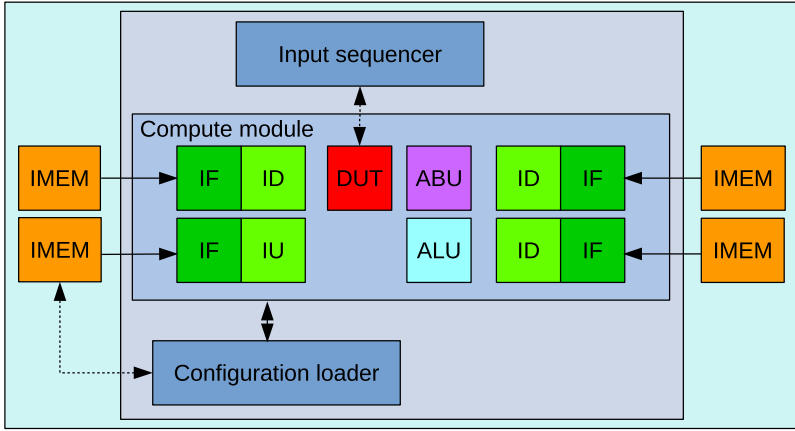


Fig. 3. Unit test structure for function units. Shown are the compute module and the memory module containing the instruction memories. The device under test (DUT) is shown in red and represents the function unit that is being evaluated.

unit testing, only the “compute module” is synthesized. Some DUTs, such as the switch-boxes, have their own dedicated test-benches, as this allows for better direct control.

This section describes how the required input vectors and application are generated, how the energy and area for function units are obtained, and how energy and area for the memories are estimated.

3.3 Input Vector and Program Sequence Generation

In digital circuits, the energy consumption depends strongly on the switching activity in the circuit. We take an inverter as an example. Every time the input value changes (toggles) the output of the inverter will also change. When this happens the capacitance of the input gates of the fan-out gates has to be either charged or discharged, thus causing a current to flow through either the NMOS or PMOS transistor. The input value of the inverter does not change instantly but has a slope. This means that there is a point at which both transistors are partially conductive, thus causing a current to flow from the positive to the negative power rails of the circuit. Due to these properties, CMOS circuits have their highest power draw when switching. This is called dynamic power dissipation and is data-dependent. In the steady state, the circuits still have some leakage through the transistors. This is called static power dissipation. Within CGRA-EAM, the static power dissipation is captured with the “nop” instructions for inactive function units.

The expectation, therefore, is that energy for the DUTs will vary significantly with the number of toggling input bits. However, the model is expected to operate on an instruction trace only. Therefore, no information about the actual data passing through the design can be assumed. This would be the case in a DSE tool that is iterating over an instruction schedule and a hardware configuration; in this case, the operations are known, but there is no information about the data values. The only assumption that can be made is that, most likely, the data values loaded by the LSUs are a reasonable representation of the data width in the data-path.

For this reason, the DUTs are evaluated for the data widths supported by the Blocks load-store units (LSUs): double-word (64-bit), word (32-bit), half-word (16-bit), and byte (8-bit). Of course, this is limited by the configured data-path width of the design. Throughout this section profiling will be performed using 32-bit function units. This requires DUT evaluation for word, half-word,

and byte values. For a byte this means that the lower eight bits will be applied to the DUT with a randomly generated value, while the upper 24 bits will be kept constant at zero.

The input values are generated as (uniform) randomly toggling individual bits for a specified width for each of the inputs of the DUT. These input values are stored as a sequence of input vectors; the input sequencer in Figure 3 provides these inputs synchronously with a Blocks application. If the DUT is a function unit, then the application is generated in parallel with the input vectors. This application contains all operations valid for the DUT. Most function units have multiple inputs and outputs; the test-bench activates all possible input and output combinations multiple times. This is done to obtain a reliable representation of the energy used by each operation. Some units, such as the ABU, can operate in multiple configurations (e.g., branching mode and accumulation mode). Since this would require reconfiguration half-way through the test sequence, these configurations are evaluated as independent tests.

The switch-boxes require a different structure. Various switch-box configurations are loaded, one after the other. Each configuration implements a number of configured paths through the switch-box; this allows to obtain energy numbers for different numbers of configured paths. This enables interpolation and extrapolation in the energy model. Each configuration is exercised with multiple input vectors, similar to the function units.

3.4 Obtaining Energy and Area Information

Typically, synthesis tools report only average power over a whole simulation run, which would require one simulation run per operation that needs to be classified. The Joules application within the Cadence **application-specific integrated circuit (ASIC)** design tool-chain can provide power per time interval. Setting this time interval to one clock period provides per-cycle power measurements. Each clock-cycle, therefore, becomes a measurement “segment” for Joules.

Obtaining power per cycle has the advantage that power can be correlated to the number of changing input bits. Although not used in the current energy model, as will be shown in Section 4, it can be used to gain more insight into why a model may be less accurate in some situations. For each evaluated DUT, and for every operation it can perform, at least 1,000 measurements were performed for each data-type. By doing so, an accurate distribution of the power draw can be obtained. Furthermore, the actual number of toggling bits is correlated such that it is possible to estimate power draw based on the number of actual toggling bits.

After power draw is obtained, these numbers need to be related to the evaluated operations. This is performed by a script that takes a trace of these operations and matches them to the corresponding cycle in the power estimation. Since the clock period is known, the power draw can be converted into energy per operation. These numbers can then be used to gain insight in the behavior of the DUT via plots such as those shown in Figure 4. This figure shows the energy per operation for the ALU. It can be observed that within the same data size the medians are quite similar, with the exception of operations such as “add” and “sub.” These operations cause rippling in the full-adders inside the arithmetic logic unit (ALU) that cause a somewhat higher energy consumption. The “nop” operation is very noticeable due to its much lower median combined with its very skewed distribution. Although the median is around 0.2 pJ per operation, there are numbers up to 2 pJ per operation reported on the higher side of the distribution. These are caused by toggling the input values even though the ALU does not write to any output register. The changing input values cause changes in the combinational logic that cause some power draw. Actively isolating unused inputs may prevent this, but causes a higher energy consumption for all operations, as well as a longer critical path.

This profiling method works well for most function units. However, the multiplier is more sensitive to the actual data input, as can be observed in Figure 5. In this case, the median values are

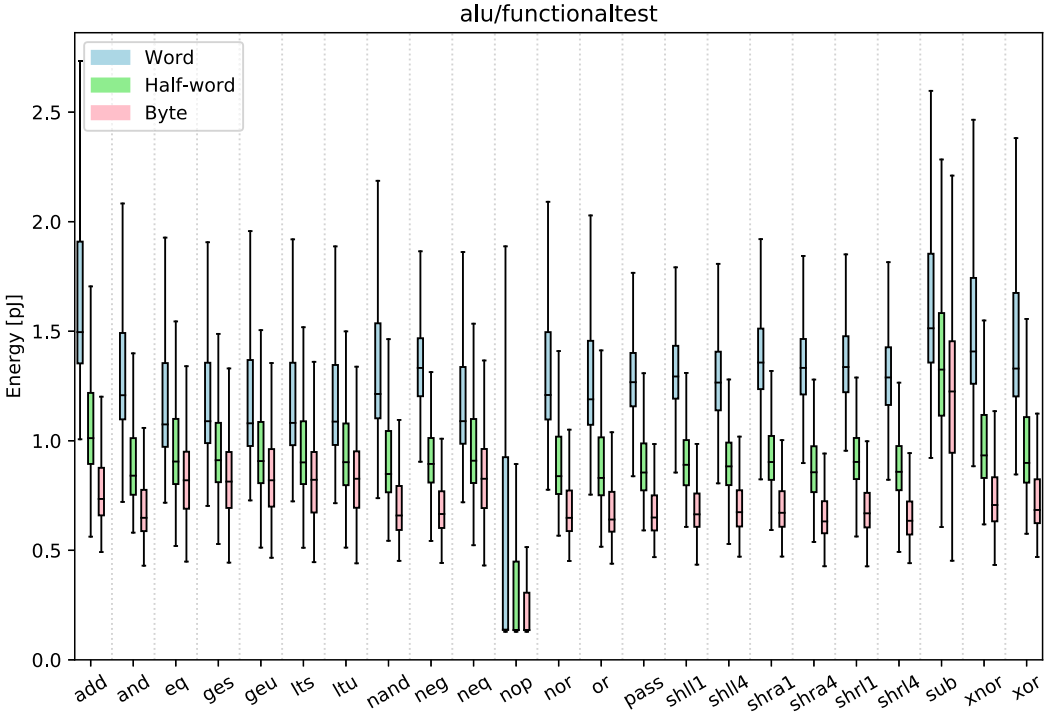


Fig. 4. Energy per operation at various data-widths as recorded for the ALU function unit, for a 40 nm low-power technology library at nominal voltage and typical temperature.

much further apart than those shown for the ALU, in Figure 4. For the multiplier it is therefore important that at least the data-type is used in the model. Without it, the error will become significant. Especially since the multipliers are likely one of the larger, and more energy consuming, building blocks in CGRAs. It is also interesting to observe that also for pass and no-operations the energy consumption is still significant. The reason for this is similar as for the ALU, there is still toggling activity in the combinational part of the circuit. The multiplier is characterized for word, half-word, and byte data-types. Due to the sensitivity of the multiplier to the actual data it may be advantageous to profile intermediate bit-widths, such as a half-byte. However, this requires knowledge of the actual data values during execution, something we do not know. If the compiler could give an expected data range or toggling rate, this could help to reduce the energy estimation error. In this work, however, we assume that these parameters are not known.

Extracting area for each DUT is significantly easier, as it does not depend on changing input values and is the same for all operations. The area is reported after synthesis and are extracted by the database generation scripts. These numbers can then be used for area estimation with the model.

3.5 Energy and Area Estimation for Memories

Memories used for Blocks are based on compiled memory macros designed for a specific application specific integrated circuit (ASIC) technology, in this case a 40 nm commercial technology library. For such technology libraries data-sheets are provided that specify power per read or write access, leakage, and area. An alternative is to use the CACTI model [20], but this model is intended for larger memory sizes, and it is inaccurate for very small memories, like those used in Blocks.

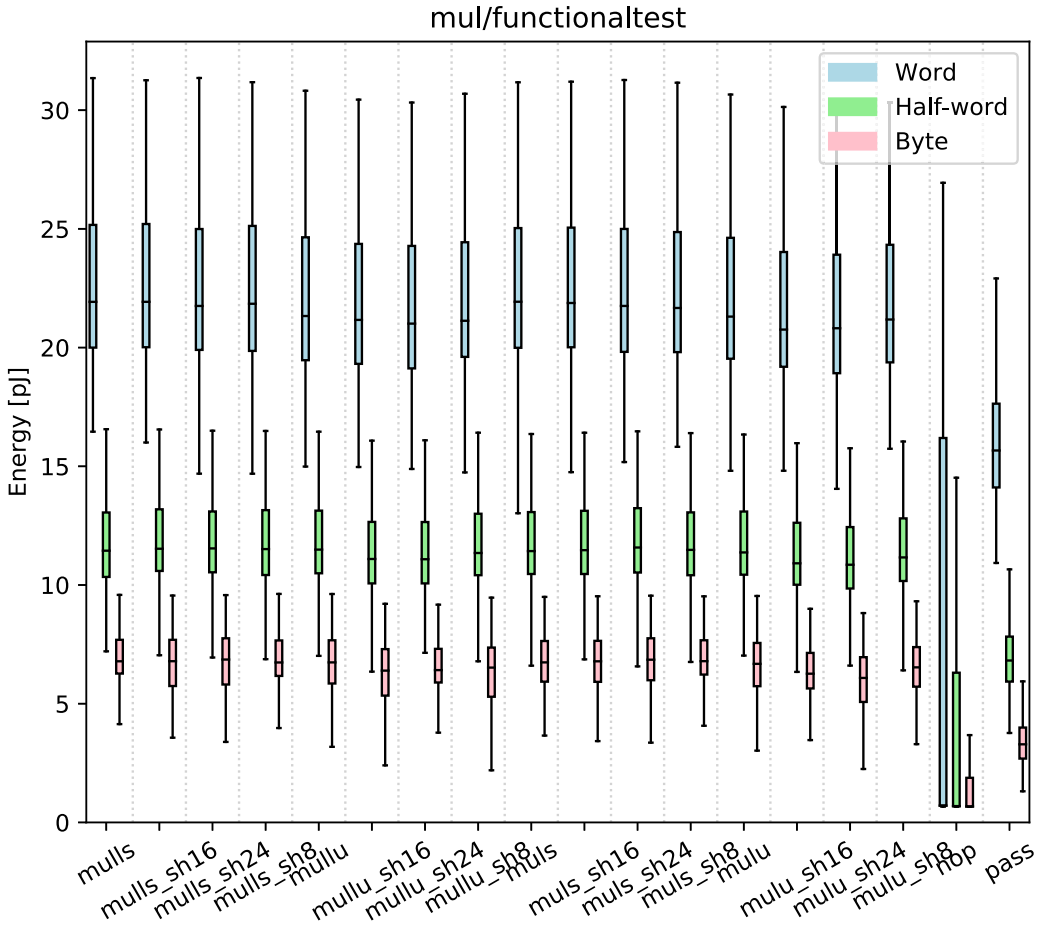


Fig. 5. Energy per operation for instructions at various data-widths as recorded for the MUL function unit.

Instead, a script extracts the relevant values for all reported memory widths and depths from the provided memory data-sheets and stores these in a database that can be used by the energy and area models.

4 PROPOSED ARCHITECTURE MODEL

The goal of an energy and area model for design space exploration is to quickly provide a reasonably accurate estimation of energy and area for CGRAs. The execution time of the model is important for automated DSE, as the model is in the critical path during design iterations. The energy and area models presented in this chapter are calibrated on the Blocks reconfigurable architecture, but the same methodology can, in principle, be adapted to various other CGRAs. The proposed model takes an application trace, or instruction schedule, as well as an architecture description as an input. Based on this information, an energy and area estimation will be made. The model is intended to provide information for both hardware resource design space exploration as well as energy-aware compilers. In such cases, no cycle accurate simulation will be performed, and the actual data that is processed is therefore unknown. This requires the model to be dataagnostic. This will introduce some error, but allows energy estimation for (partial) instruction schedules or

hardware changes without fully simulating these in advance. To keep the estimation errors limited, the data types used in the schedule are used to improve model accuracy.

Evaluation of the model is automated, as a Python script. It reads the model database, processes the trace, and computes the results. The model is integrated into the Blocks framework. Once the user has designed an application, the model can be called with a single command. The goal is to integrate the model with the design space exploration tools that are currently in development.

4.1 Model Requirements for Design Space Exploration

For CGRAs there are two aspects of an architecture instance that can be modified at design time. The first is the hardware structure, e.g., which function unit types are present in the design and how they are connected. The aspect that can be modified is an application that is scheduled on the hardware by a compiler. These two processes are not independent from each other. For example, during instruction scheduling a bottleneck becomes apparent on a specific function unit. The compiler could then decide to use another function unit of the same type. However, when the goal is energy efficiency this may not be the best choice. Such a decision can only be evaluated when an estimation can be made of the cost of adding another unit. This can be done with a model that estimates the energy change. Although it is desirable to have a model that provides accurate absolute numbers, it is more important that the scaling is correct. E.g., when doubling the number of function units, we would expect the power consumption to roughly double.

The cost functions for DSE tools usually operate on scalar values, not on probability distributions. For that reason the median for a certain data-type in the model database is used. The median value was chosen over the average, as it is less biased towards outliers. As can be seen in the box plots in Figures 4 and 5, the outliers are not always evenly distributed on both sides of the box. In case the worst-case energy consumption is desired, it is possible to take the maximum instead of the median. However, for DSE where the model is used to make decisions based on scaling, this may lead to unwanted effects when the variation is very large.

As the goal is to evaluate (partial) instruction schedules, it is not always possible, or desirable, to perform a cycle-accurate simulation that includes actual input values. In other words, the actual data values are unknown. However, the data-type is known; e.g., we know if a word or byte is used to represent a variable. These properties lead to the following requirements on the model:

- (1) A model that provides reasonable absolute numbers and accurate scaling properties.
- (2) The output of the model must be usable as an input for DSE tools; e.g., a scalar value that can be used in a cost function.
- (3) The model shall be data-agnostic, as actual data values may not be known during DSE.
- (4) The model must provide a breakdown of area and energy consumption by function unit type, allowing targeted optimization during DSE.

4.2 Energy Model

The energy model takes the architecture description, the application code (**parallel assembly (PASM)**), and a trace of the program counter as an input. Another input for the model is a database with energy-related properties for each building block, as shown in Figure 6. This database is ASIC technology-dependent. This means that for each of these technologies, or specific temperature and voltage corners thereof, the Blocks energy, area, and timing properties need to be evaluated as a one-time effort.

The architecture and application code are part of a Blocks configuration. A program counter trace can be obtained either by simulation or based on polyhedral models such as Polly [5]. The

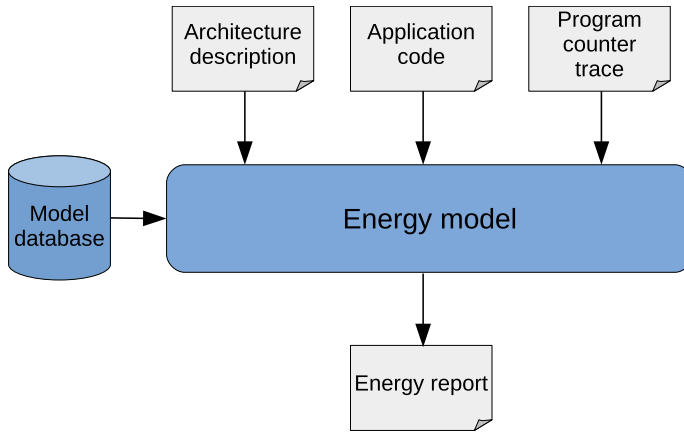


Fig. 6. Inputs and outputs of the energy model.

trace is required by the energy model to evaluate how many times each branch is taken. The energy model follows the program counter trace and looks up which line of instructions is executed in the application code. If there is an LSU operation that performs a load or store on one of the memories, then this determines the assumed data width for all instructions afterwards, up to the next load or store.

This can be done accurately, since the load and store instructions have the data-type specified as an argument. For example, a memory load could be LGA BYTE, out0, in0. This instruction performs a load on the global memory, taking the memory address supplied at input 0 and stores the value at output register 0. The data-type is also given, as the LSU will realign the data to the specified type. This specified data-type is then used as the data-type for all future computations.

In Blocks, the compute instructions are data-type agnostic. However, for architectures where this is not the case there will be multiple variants of the same instruction, e.g., a MUL BYTE instruction. These instruction variants can be profiled and used in the model without requiring any changes.

Although this is not as accurate as performing a functional simulation and generating a trace of the actual encountered data widths, it allows the model to be used without any assumption on the data other than the information provided in the instructions. This is typically also the case in compilers; the actual data values are often not known but the data types of their variables are. The total energy for a Blocks architecture instance is computed by Equation (1):

$$E = E_{array} + E_{arbiter} + \sum_{\forall \text{ memories}} E_{memory}, \quad (1)$$

where E_{array} indicates the energy consumption in the (reconfigurable) array, this includes the function units, instruction decoders, and switch-boxes. The energy consumption of the memory arbiter is described by $E_{arbiter}$, and the energy for each of the memories (local and global) is indicated with E_{memory} . As will be shown in the remainder of this section, each of these terms can again be broken down in smaller parts.

4.2.1 Computing Energy for Function Units. While traversing the instruction trace, the occurrence of each operation is counted. This is done for each individual instruction decoder, which has one or more function units under its control. To determine which function units are under control of an instruction decoder, the paths routed over the switch-boxes have to be analyzed. This is performed as a pre-processing step in the energy model. Depending on the configuration, it is very

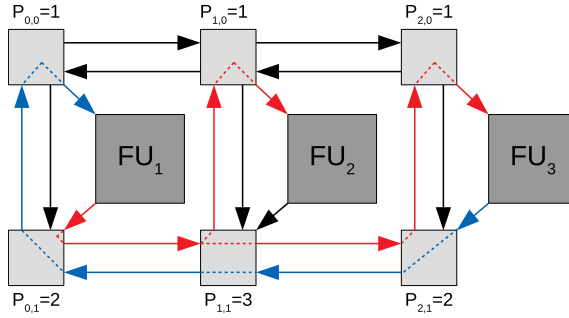


Fig. 7. Example showing active paths through the switch-boxes. The red and blue lines are paths that can be activated with operations other than a no-operation. The paths are specified in the configuration bit-stream.

well possible that not all function units are used. In Blocks, unused function units have their data inputs and control inputs tied to a zero value. This results in these units effectively performing a no-operation. The estimated energy is somewhat pessimistic in this case, since the model assumes toggling inputs, while in reality these are tied to a fixed value.

For each instruction decoder and its associated function units, the energy for every operation is added to form the total energy consumed (see Equation (2)). This includes the unused units that are assumed to perform no-operations. In this equation, E_{ID} indicates the energy consumed by the instruction decoder for each performed operation. This instruction decoder can control multiple function units as an SIMD unit; this number is indicated with N_{FU} . The energy consumed in a function unit depends on the executed operation and on the datatype of the operands. This is indicated with the function $f_{FUenergy}(dtype, oper)$, which essentially performs a look-up in the model database. To compute the final result for the (reconfigurable) array, the energy dissipated by the switch-boxes still needs to be added, which is indicated with $E_{switchboxes}$ in Equation (3).

$$E_{compute} = \sum_{\forall operations} E_{ID} + f_{FUenergy}(operation, datatype) \cdot N_{FU} \quad (2)$$

$$E_{array} = \sum_{\forall compute} E_{compute} + E_{switchboxes}. \quad (3)$$

4.2.2 Computing Energy for Switch-boxes. The energy consumed by the (circuit switched, statically configured) switch-boxes depends mostly on the number of active paths through the switch-box. For the switch-boxes in the control network, this depends on the paths from instruction decoder outputs to function unit instruction inputs. These paths are determined by recursively following the output of each instruction decoder through the switch-boxes to their destinations. By doing so, a total number of “activated” switch-boxes can be determined for each instruction decoder output change. Every operation other than a **no-operation (nop)** is considered a change on the instruction network.

Figure 7 shows an example network of three function units and six switch-boxes. The arrows indicate configurable paths between switch-boxes and function units. The red and blue paths are configured paths that can be activated if an operation other than a “nop” is executed. For each switch-box, the number of configured paths is indicated with $P_{x,y}$. This is the maximum number of active paths through that switch-box at horizontal location x and vertical location y . It can be observed that for the switch-box with $P_{1,1}$ the number of paths is three. This is because the red path splits to provide data to two separate outputs. Effectively, each used output is considered to

be a path, as it requires configuration (and indeed a path) through the multiplexers that make up the switch-box.

The producer of data for the red path is FU_1 , the producer for the blue path is FU_3 . In case that FU_3 executes a no-operation and FU_1 an “active” operation, only the red path is considered activated. In this case the number of active paths is two for the switch-box at location (1,1). Based on the energy consumption for the benchmarked switch-box configurations, the energy is predicted for the switch-box in the architecture instance. In case the exact number of active paths is not in the model, the energy is interpolated or extrapolated depending on the available data in the model.

The switch-box energy can be calculated with Equation (4). The function f_{energy} performs a look-up in the model for a given number of active paths (P_{active}) and type of data passed over this channel in a given cycle (*datatype*). If required, then this function performs interpolation or extrapolation. Since switch-boxes have some static power dissipation, meaning that there is energy consumed even if the switch-box is idle. This is captured with E_{idle} .

$$E_{switchboxes} = \sum_{\forall \text{ switch-boxes}} f_{energy}(P_{active}, \text{datatype}) + E_{idle} \quad (4)$$

4.2.3 Computing Energy for the Arbiter. An arbiter is always difficult to model, due to its complexity, like performing coalescing, and CGRAs are no exception in this respect. The arbiter that handles access control for memory operations issued by the LSUs is instantiated with a number of request ports equal to the total number of LSUs in the design. Arbiters with varying numbers of ports are characterized in the model database. For this evaluation, arbiters with 1, 2, 4, 8, and 16 ports were characterized. If an arbiter size is not present in the database, then the energy for a certain operation is interpolated between the two closest arbiter sizes or extrapolated if the arbiter size is larger than those available in the model database. This interpolation does, of course, introduce an error. However, this is not an inherent limitation of the model or the methodology: If a higher accuracy is required, then it is possible to characterize more design points of the arbiter, e.g., all port numbers that are currently not in the model database.

The number of cycles it takes to complete all access requests depends not only on the number of requests but also on whether the requests can be coalesced into fewer memory operations. As the model performs no data analysis, it cannot tell, only based on the operation issued by an LSU, whether an access is going to be coalesced or not. However, when performing simulation the program counter trace shows how many stall cycles occurred after each set of access requests. A compiler, which has knowledge of the access patterns, could provide the same information. The energy consumption for the arbiter (which is active even during stall cycles) is slightly different for stalled and non-stalled cycles. This is due to the initial detection that access requests can be performed in parallel. Figure 8 shows the energy per operation for an arbiter with four request ports. Some operations are shown multiple times: for uncoalesced (u), coalesced (c), or broadcast (b) accesses. Operations that caused the the arbiter to insert a stall cycle in the processing pipeline are marked with (s). Between coalesced and uncoalesced memory accesses is some energy difference. But, since it is now known in the model whether accesses are coalesced, the assumption is made that the programmer or compiler will attempt to achieve coalesced accesses where possible.

In Figure 8 it can be observed that, for the arbiter, the data type does not make much difference to the consumed energy. This is the case for all evaluated arbiter sizes and is likely caused by the fact that the data loaded from the memory contains non-zero bits over the full data-width. The active bits for each data type are only selected at the LSU, and not yet in the arbiter. Since the energy consumption is so similar, the energy model of the arbiter does not take into account the data type of these operations in the look-up function $f_{energy}(\text{operation})$ in Equation (5). Furthermore, the idle

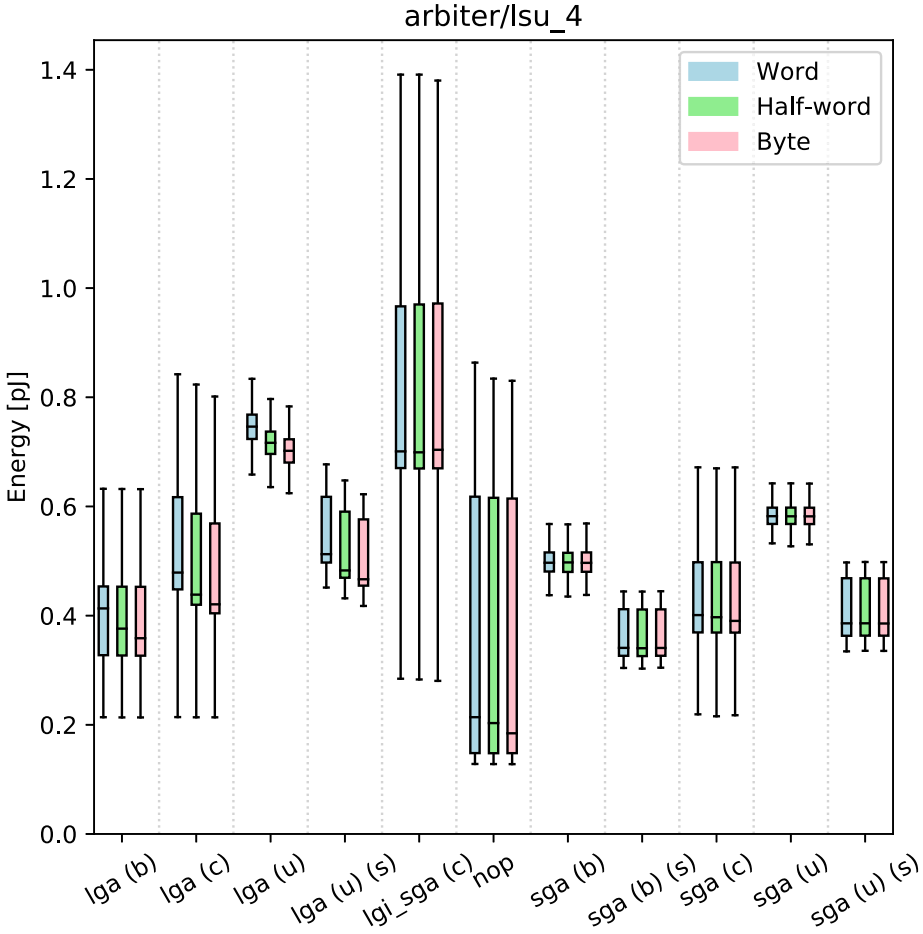


Fig. 8. Energy for operations performed by an arbiter with four request ports. It can be observed that the data type is relatively unimportant. Several operations can be observed here: lga and sga are explicitly addressed global load and store operations; lgi is an implicitly addressed (automatic address generator) load operation. Some operations can be executed in parallel, such as lgi_sga, an implicit load in parallel with an addressed store operation.

energy of the arbiter is indicated with E_{idle} .

$$E_{arbiter} = \sum_{\forall operations} f_{energy}(operation) + E_{idle} \quad (5)$$

4.2.4 Computing Energy for the Memories. The memory energy consumption is directly based on the manufacturer datasheets for memory macros that are available for the used ASIC technology. This was done because no direct access to the SRAM compiler was available. The datasheets describe some of the most common memory configurations, including the maximum width and depth, which can be provided for a certain memory type.

In the Blocks architecture, two memory types are used: dual-ported register files (for local memories and instruction memories) and dual ported SRAM (for the global memory). In both cases, the memories are so-called “simple” dual ported memories, where one port can only read and the other port can only write.

To estimate memory energy, the accesses to each of the memories are counted. For the global memory, this means counting global load and store operations. These take at least one cycle but may take more in case multiple memory accesses are required. Therefore, the number of memory accesses is counted as shown in Equation (6):

$$N_{GMaccesses} = \sum_{\forall GMoperations} \left(1 + N_{stalls}^{GMoperation} \right), \quad (6)$$

where N_{stalls} is the number of stalls caused by the **global memory operation (GM operations)** issued to the arbiter. Reads and writes are counted separately to compute energy, using the same equation. Although the processor is inactive during a stall cycle, the memory accesses causing this stall are still being executed. For this reason, stalls are counted as memory accesses.

For local memory accesses, there is no arbiter between the LSU and the memory. Therefore, no stalls occur and the number of accesses is equal to the number of times a local load of local store operation is performed. For the instruction memories, the number of read accesses is equal to the length of the instruction trace.

With the number of accesses known, the energy for each memory can be computed using the energy consumption in the energy model database according to Equation (7). In case the exact memory size is not present in the database, either in the number of entries or in the width of the memory, interpolation or extrapolation are used to estimate the energy consumption.

$$E_{memory} = (N_L \cdot E_L) + (N_S \cdot E_S) + (N_{notIdle} \cdot E_{static}) + (N_{idle} \cdot E_{idle}) \quad (7)$$

The energy for loads (E_L) and stores (E_S) are different and are multiplied with the computed number of loads (N_L) and stores (N_S). On top of the access energy there is also some static energy overhead. Furthermore, when the read and write ports of the memory are disabled (when the memory is idle) there is still a leakage current that causes energy consumption when no loads or stores are performed (E_{static}). This current is lower than the static power dissipation for enabled memories and is, therefore, counted separately. The idle energy for an operation is multiplied with the number of cycles that the memory is idle (N_{idle}). Unused memories, for example local memories of an unused LSU, are considered idle for the length of the instruction trace.

4.3 Area Model

The area model is much simpler than the energy model, since it is not required to obtain any activity information on the function units or the operations they perform. The area model, therefore, only requires the architecture description to obtain information about how many function units, memories, and switch-boxes are present. Figure 9 shows the flow to obtain area information.

The array area can be obtained by simply adding up the area for the number of function units (N_{type}) of each type ($type$). The function $f_{area}(type)$ performs a look-up in the model database for the area of each function unit type. If present, then the area (A_{swb}) for the switch-boxes in the control and data network is added to the array area, multiplied by the number of switch-boxes in each of these networks (N_{swb}). This leads to Equation (8).

$$A_{array} = \sum_{\forall type} \left(f_{area}(type) \cdot N_{type} \right) + A_{swb}^{data} \cdot N_{swb}^{data} + A_{swb}^{control} \cdot N_{swb}^{control} \quad (8)$$

The area for the arbiter depends on the number of channels; this number is equal to the number of LSUs present in the design. If no arbiter is present in the model database that has the same number of request channels, then the area is interpolated or extrapolated to estimate the actual arbiter area.

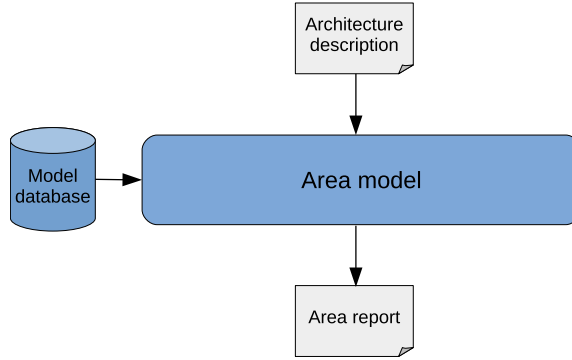


Fig. 9. Inputs and outputs of the area model.

Memory area is, similar to the memory energy, obtained from the memory datasheets. These numbers are provided for a finite set of memory configurations (depth and width). This often requires interpolation or extrapolation on the depth, width, or both to obtain an estimate for the memory area.

The total area for a Blocks architecture instance is computed according to Equation (9). This ignores the area for smaller modules in the design, such as the configuration loader and communication interfaces. However, the total area of these is comparatively small to the area of even a single function unit.

$$A = A_{compute} + A_{arbiter} + \sum_{\forall memories} A_{memory} \quad (9)$$

5 EVALUATION

For the model to be usable for design space exploration (DSE), it needs to predict energy and area numbers close to the actual numbers. In this section, evaluation of the model is performed. As a reference, the numbers reported by the ASIC synthesis tool are used. These numbers are based on the same 40 nm technology library as used for micro-benchmarking the function units. For evaluation, eight benchmark kernels are used. These benchmarks all come from the signal processing and computer vision domain.

The Blocks architecture flow allows instantiation of hardware with the reconfigurable network replaced by fixed wiring between function units, and all unused function units are removed. These hard-wired versions, unique to each benchmark kernel, are also evaluated to differentiate between errors introduced by modeling the reconfigurable network and those introduced by modeling the function units. These hard-wired architectures are referred to as “Blocks-ASP” (**A**pplication **S**pecific **P**rocessor).

The benchmark kernels are from the computer vision and signal processing domains and include both 1D and 2D operations. The kernels are: (1) Binarization, (2) Erosion, (3) Projection, (4) FIR filter, (5) IIR filter, (6) FFT, (7) 2D-convolution, and FFoS (part of an industrial vision application [18] using a combination of kernels 1, 2 and 3).

5.1 Energy Estimation for Hard-wired Instances

Figure 10 shows the reference and estimated energy numbers for hard-wired Blocks instances (without switch-boxes) that essentially form application-specific processors. Each benchmark kernel has its own hard-wired architecture, generated using the Blocks framework. It can be observed that for all benchmark kernels except “2D-convolution” the predicted energy numbers are very

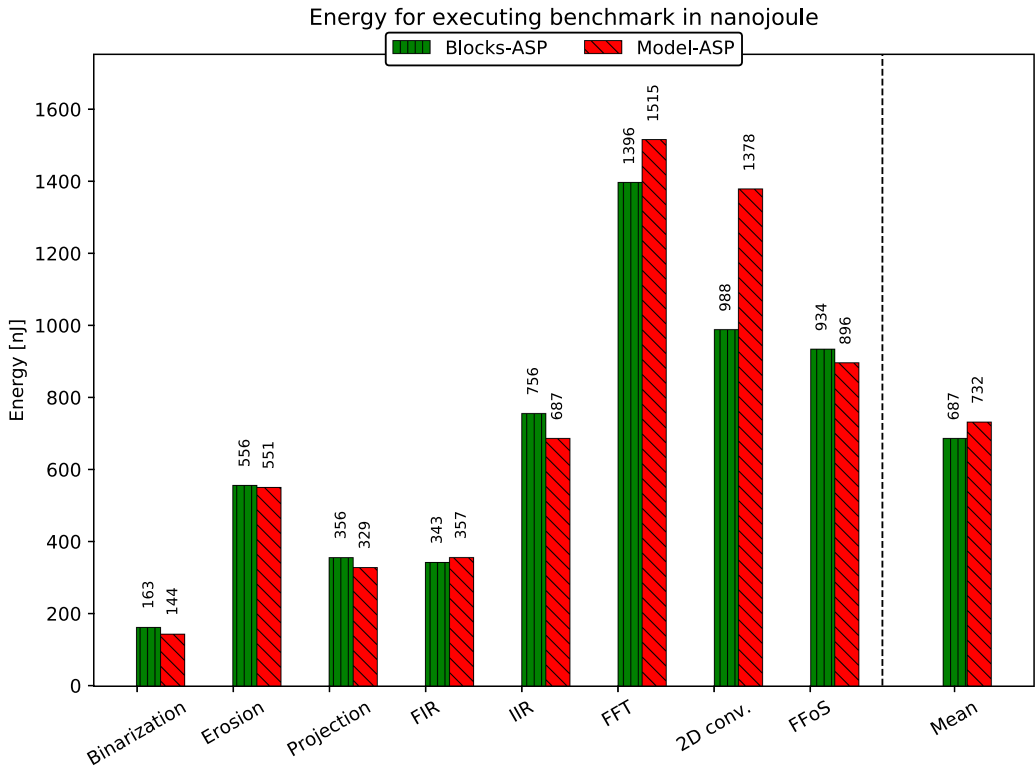


Fig. 10. Energy reported by ASIC synthesis versus energy reported by the energy model for Blocks with hard-wired connections.

close to the reference numbers. Not taking “2D convolution” into account, the error of the model lies between -12% for “Binarization” and $+4\%$ for “FIR.”

The energy for the “2D convolution” benchmark is 39% overestimated by the model. This is due to the data-type used in the application in relation to the actual values. The benchmark performs a 3×3 convolution on an image. The values of this image are loaded as bytes and range between 0 and 255, using the entire range of the byte data-type. However, the convolution coefficients range between 0 and 2. Furthermore, the convolution is implemented as a spatial layout using nine multipliers. The convolution coefficients are, therefore, entirely static during program execution. The energy estimation in the model is based on a multiplication of 8-bit by 8-bit values, while in reality only 8-bits can toggle in this benchmark application. Figure 5 shows the cost of different multiplications for a varying number of bits. For the specific multiplication performed in this benchmark (mullu_sh8) the median energy is 6.6 pJ per operation. But, since most of the coefficients are either zero or one, the actual energy value is closer to a “pass” operation. Such an operation simply forwards the input value to an output. The energy median for the “pass” operation is 3.2 pJ per operation. The total energy contribution of the multiplier units in this benchmark is 469 nJ, which is 34% of the total predicted energy. If the multiplier energy is scaled to 48% of its original value (the ratio between the multiplication and pass operations in terms of energy), then the multiplier energy becomes 227 nJ. This brings the total energy for the benchmark to 1,136 nJ and reduces the overestimation of energy to 15%.

The “2D convolution” benchmark shows that although in most cases it is sufficient to compute energy based on the data types used, there are cases in which the actual data values can have a

significant impact on the estimated energy consumption. However, extending the model to take this into account would make it more complex and slower. If this level of accuracy is required, then it may be possible to have a compiler, generating code for the architecture, provide an estimation on the actual data width instead of just the data types. Especially in cases where values are constants, such as the coefficient value in the convolution benchmark, it should be possible to provide estimations on this. If the model is used for DSE, then such accurate absolute energy estimations are probably not required; the relative changes in energy between design iterations are more important for the cost functions used in such tools.

Besides providing an accurate measure of the total energy of an architecture instance, it is important to also estimate the individual contributions of parts of the design. This allows a designer (or automated tool) to gain insight into what part of the design is consuming the most energy. Figure 11 shows an energy breakdown, based on the post-synthesis power estimation provided by the ASIC toolflow for Blocks (marked with “B”). The energy breakdown is divided into several categories: the arbiter, the function units, the **global memory (GM)**, the **instruction memories (IM)**, the **local memories (LM)**, the switch-boxes (if present) for both control and data networks (SWB), and other parts of the architecture that cannot be classified as any of these categories. The “other” category includes communication bus interfaces, boot-loader, configuration scan-chain, and wrappers around memory blocks (for bypassing and automatic power-down). It also contains gates that are optimized out of the original module hierarchy by the synthesis tool and can no longer be linked to a specific module. It is of course possible to force the synthesis tool to keep the hierarchy, but this prevents some optimizations from being applied. Since this part of the evaluation is based on the hard-wired Blocks architectures, there is no energy consumption for the “SWB” category reported in this figure.

The results between the synthesized Blocks instance and the model are somewhat skewed due to the presence of the “other” category. If this category would be added to the model results (or be removed from the synthesis results), then the trends between both methods are very comparable. As the trend is more important than the absolute values for DSE, the breakdown satisfies the first and fourth requirement of the model.

Figure 11 also shows the estimated energy distribution provided by the energy model (marked with “M”). When comparing the estimated energy consumptions to those provided by the ASIC tools it can be observed that the energy estimations for the function units and the global memory seem to be quite accurate; with the exception of the “2D convolution” and “FFT” benchmarks, where the multiplier energy, and thus the function unit energy, is overestimated.

The energy contribution for the arbiter seems to be underestimated; this may be due to the fact that for benchmarking the arbiter was evaluated separately from the rest of the architecture (like all the architectural building blocks tested). However, in a real architecture, the arbiter connects to LSUs that are spread throughout the design. It is possible that these location differences cause larger buffers to be selected in the data-path, during synthesis, to meet timing requirements, leading to a higher energy consumption. The underestimation is largest for the “binarization” benchmark. In this benchmark, the computation per byte of data is very low, leading to a high number of memory requests relative to the total number of cycles. Binarization is also the benchmark where the total underestimation of energy is the largest for all benchmarks. At least part of this is due to arbiter energy estimation. A possible solution for this is to scale the arbiter energy with the total area of the design.

For the local memories, the energy contribution seems to be somewhat underestimated. Part of this can be attributed to underestimation of leakage. This can be seen in, for example, the “binarization” benchmark, where the local memories are not used. Where the synthesis report shows some energy consumption in the local memories, the model hardly shows any energy contribution for

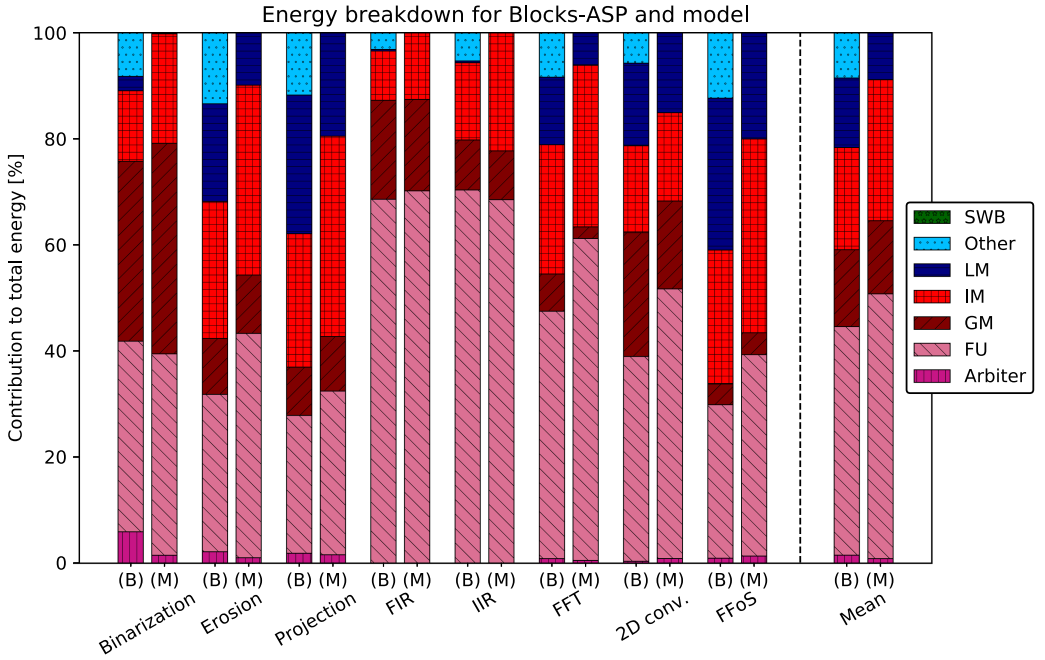


Fig. 11. Energy breakdown for the hard-wired architectures, according to the synthesis reports (B) and the model (M). Since the hard-wired architectures do not have a reconfigurable network, the contribution of switch-boxes to energy is zero.

the local memories. As the energy figures for the memories are directly based on those provided in the datasheet, it is not clear what causes this difference. One cause might be that the data lines to the memory macros are currently not isolated, meaning that for some instructions these lines may be toggling even when no data is actually written to the memory. Both for real energy, as for estimated energy, it may be good to provide active isolation of these signals. However, this does increase the length of the path from LSU to local memory. If also applied to the global memory, then the isolation will be inserted in the critical path of the design, which should be avoided.

However, the energy for the instruction memories seems to be overestimated in the model. This is most likely caused by the fact that in the model every instruction is considered to be a change of value on the instruction memory interface. In reality, however, instructions do not always change between cycles. For example, an application may be designed to provide a loop-body where the instruction is (almost) static. Although the instructions are still fetched from the instruction memories, the actual values on the instruction memory bus do not change.

A DSE tool will steer, to a large extent, design space exploration on function unit energy consumption. The mean absolute prediction error for this category is 20.6%. However, the absolute number is less important than the scaling of the results when a function unit is added or removed.

5.2 Energy Estimation for Reconfigurable Instances

The reconfigurable architecture used to evaluate the benchmark kernels is the same for all kernels. For this reason, it contains the superset of function units required in the hard-wired architectures. The structure of the instantiated reconfigurable instance is the same as used in Reference [23], which is shown in Figure 12.

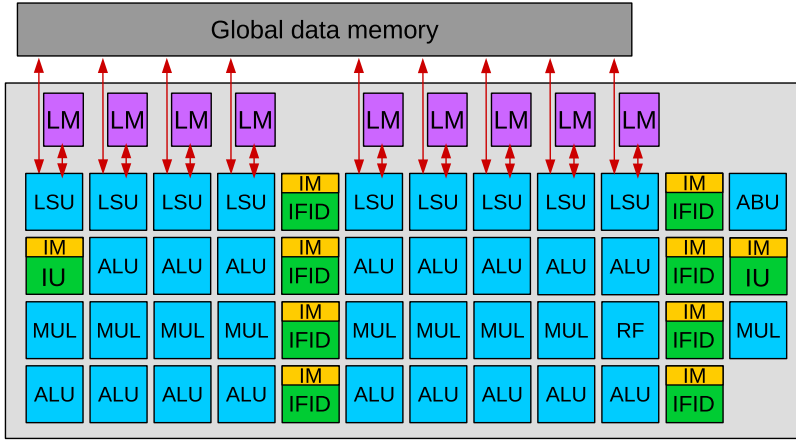


Fig. 12. The reconfigurable architecture used for evaluating model accuracy with respect to post-synthesis simulation and power estimation. The details behind this architecture instance can be found in Reference [23]. The function units are shown in blue; these function units are interconnected over a mesh-like interconnect for data transport. Furthermore, a second network is used to connect function units to instruction decoders (green), which in turn are connected to instruction memories (yellow). Load-store units (LSU) can access both local memories (purple) and global memory (gray).

Estimating energy for the reconfigurable architecture instances includes estimating energy for unused function units and memories as well as the reconfigurable switch-box networks. As the switch-box energy makes up a reasonable fraction of the total energy consumption of the design, accurate estimation thereof is important. Figure 13 shows the reference and estimated energy consumption for reconfigurable Blocks architecture instances.

Similar to the hard-wired application specific designs, there is some under- and overestimation of energy. The “2D convolution” benchmark is still overestimated, but due to the higher total energy, the overestimation is reduced to 29%. A new interesting case is the “FFT” benchmark, which is underestimated by approximately 19%. The other benchmarks have estimation errors between -20% and $+14\%$. In general, the model, therefore, has a tendency to underestimate a little.

The reason that the error margins on reconfigurable architecture instances are higher can be explained by the fact that the energy consumption of the switch-boxes, and therefore the networks, is highly data-dependent, much more so than for the function units and memories. This is because the contribution to the total energy of the data-paths is, percentage-wise, much higher for switch-boxes than for function units (as these have more control logic). This means energy differences due to data streams that are not behaving in a similar way as during micro-benchmarking are amplified in the reconfigurable networks. This is true for, for example, the “FFT” benchmark. When comparing the breakdowns shown in Figure 14, it can be observed that the switch-box energy is the cause of the underestimation. In fact, the “FFT” benchmark has the highest switch-box energy for all evaluated benchmark kernels.

The cause of the underestimation lies in the data behavior of the input data and intermediate data that is passed between function units. The “FFT” benchmark operates on two’s-complement numbers close to zero, as the input signal is constructed from two added sine waves. The results are bytes, stored as signed words. The sine waves cause the signal to move above and below the zero value. In two’s-complement notation, negative numbers have all leading bits set to one and positive numbers have all leading bits set to zero. Due to the sign extension from byte to word, this causes the highest 24 bits to alternate between all ones or all zeros. This causes a higher number

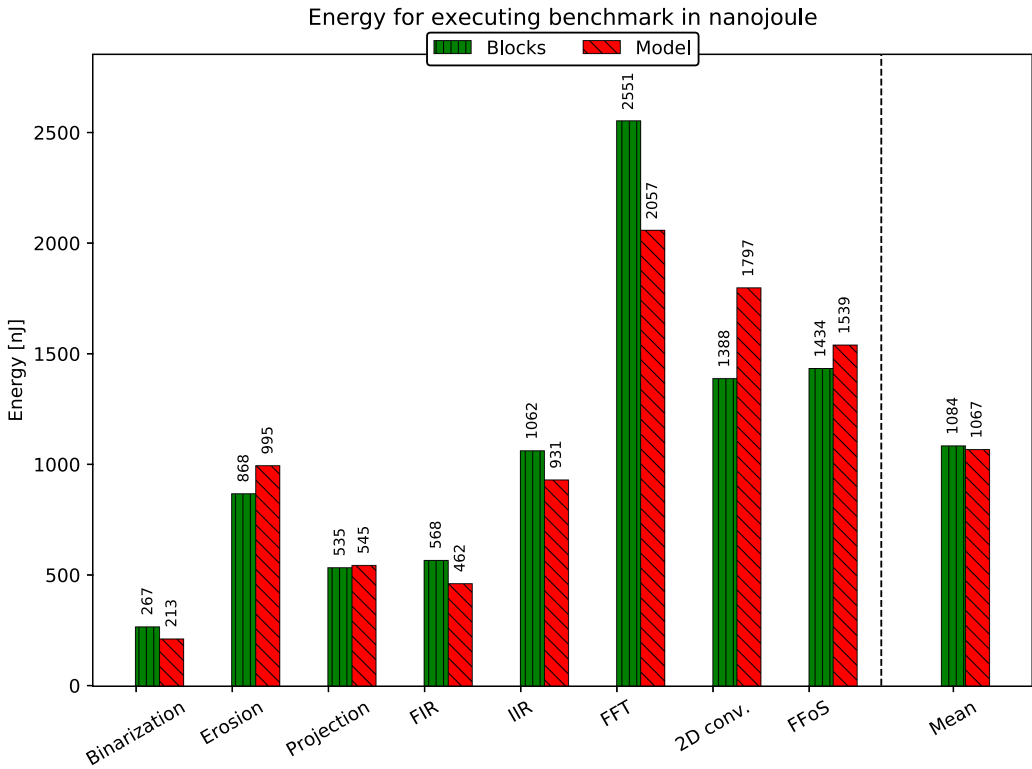


Fig. 13. Energy reported by ASIC synthesis versus energy reported by the energy model for Blocks with reconfigurable connections.

of toggling bits than the energy information in the energy model database is based on. The input vectors for the micro-benchmarks were constructed using independent randomly toggling bits. Therefore, an average of 16 toggling bits can be expected per cycle. While in the case of data for the “FFT” benchmark, this number is higher, leading to a higher switch-box energy in reality.

Figure 14 shows the energy distribution over the various parts of an architecture instance. Since reconfigurable architectures are compared, energy is consumed by the switch-boxes. Function unit energy is still very comparable to the estimates for the hard-wired architectures. Due to the presence of extra function units, which may be unused for some benchmarks, the area of the design is larger. When observing the actual and estimated energy fraction for the arbiter, it shows that for reconfigurable architectures the underestimation is more significant than for the hard-wired architectures. This supports the hypothesis that the energy of the arbiter grows with the area of the design.

The local memory energy seems to be more underestimated compared to the hard-wired architectures. When the energy consumption reported in the post-synthesis report is compared for unused local memories, their energy consumption is about a factor two to three higher than the numbers based on the memory datasheets. This may indicate that there is more activity in the actual design than is assumed in the evaluation for the datasheet (e.g., no clock gating).

For the reconfigurable architectures, a DSE tool steers design space exploration largely on function unit energy consumption and energy consumed in the reconfigurable network. The mean absolute prediction error for these two categories are 18.4% and 26.1% for the function units and reconfigurable network, respectively.

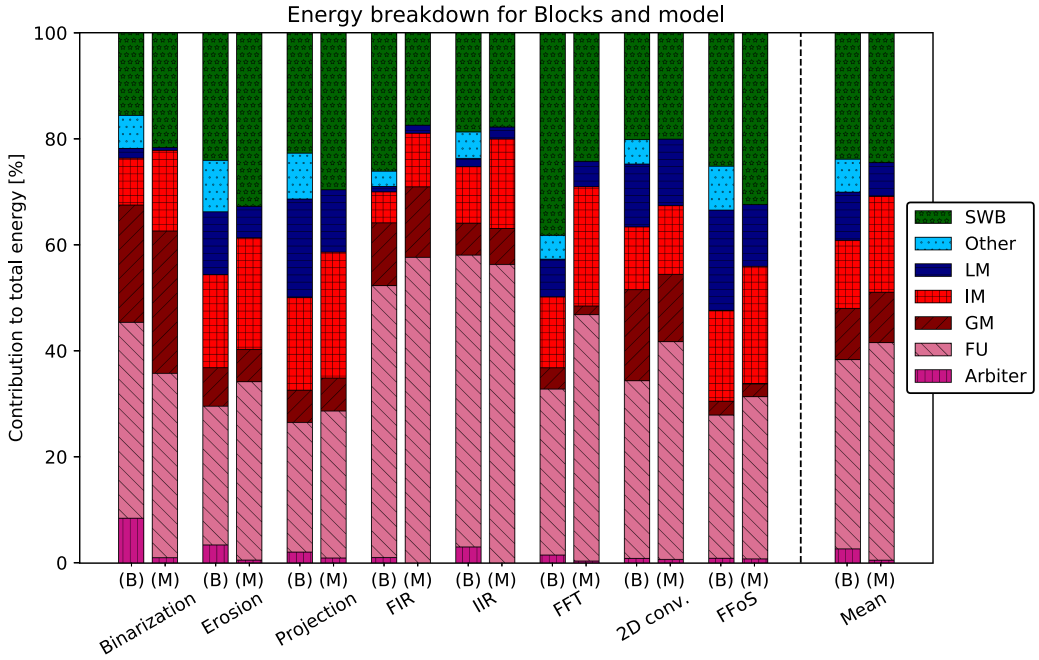


Fig. 14. Energy breakdown for the reconfigurable architectures, according to the synthesis reports (B) and the model (M).

5.3 Area Model Evaluation

As there are fewer variables at play in the area model, it may be assumed that estimating area should be more accurate. However, this is not necessarily the case, especially for the hard-wired architecture instances. Figure 15 shows the reported and estimated area numbers for the hard-wired architecture instances. The error of area estimation varies between -4% for “erosion” and $+4\%$ for “Binarization.” Other benchmarks with higher errors are “FFoS” and “Projection.”

For two of these benchmarks (“Erosion” and “FFoS”) the model underestimates the actual area. These benchmarks do not use all inputs and outputs of the available function units. The unused inputs and outputs, in hard-wired architecture instances, are unconnected or tied to ground for outputs and inputs, respectively. During synthesis, the unconnected nets are detected and removed. For outputs, this means that an entire output, including its multiplexer and buffer, can be removed. For inputs, something similar is performed, although in this case the constant values are not removed, but the logic they drive is replaced by another constant value. This iterative optimization step can result in entire inputs and their multiplexers to be removed from the net-list. Of course, this makes sense from a synthesis point of view, but it makes area estimation in these cases somewhat harder. It would be possible to extend the model with an option that compensates for the optimization of one or more inputs or outputs, but this would increase the model complexity. Currently, the area model does not process any information regarding the connections between function units and whether they are actually in use. However, it is possible to compute the number of inputs and outputs that are in use.

The area breakdown in Figure 16 shows that this is at least partially the case, but that there is more to the story. The area for the instruction memories is overestimated. For these instruction memories, the area numbers are directly related to those in the datasheet. The reported area is directly obtained from those reported by the synthesis tool, which in turn are obtained from the

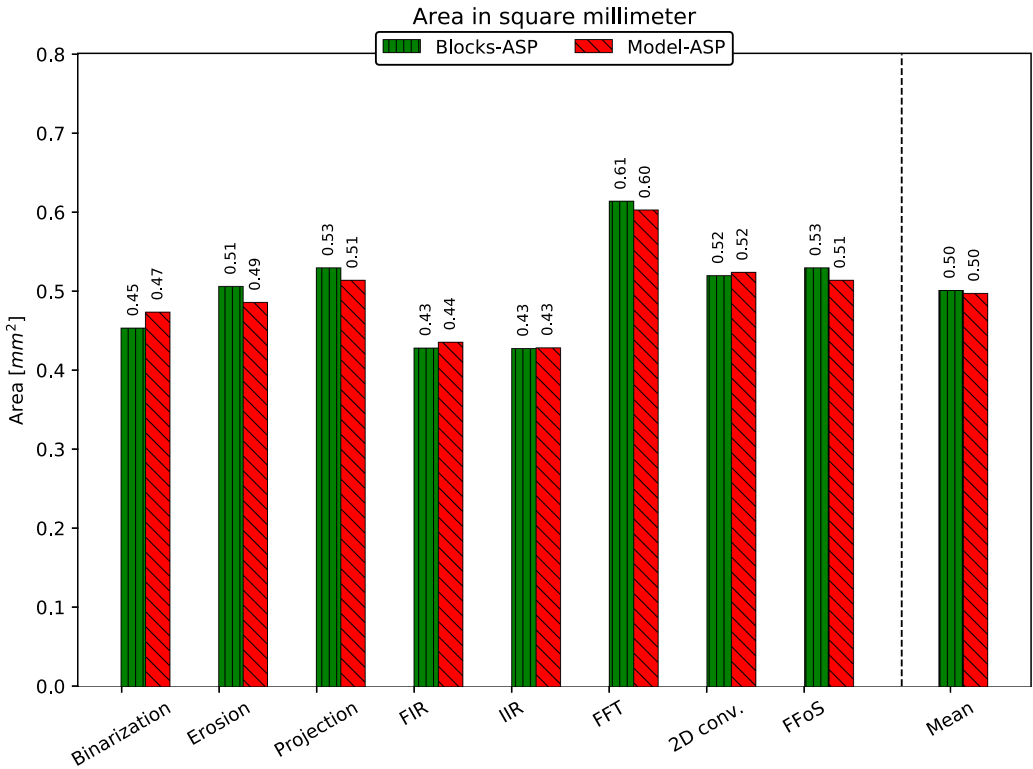


Fig. 15. Area reported by ASIC synthesis versus area reported by the area model for hard-wired connections.

memory macro description. For this reason, it is not clear what causes this discrepancy. However, the **mean absolute percentage error (MAPE)** over the benchmark set is 2.5%. For the individual categories, the MAPEs are: 10.7%, 2.4%, 16.6%, and 4.6% for function units, global memory, instruction memories, and local memories, respectively.

The area for the reconfigurable architecture instances can be accurately estimated, as all the inputs and outputs of the function units are connected to switch-boxes. The paths through the switch-boxes are configured at runtime. As the select signals of the internal multiplexers are therefore unknown during synthesis, it is not possible for the synthesis tool to determine which paths are unused or unused. Due to this, there are no paths or logic that will be removed, causing the area numbers in the model database to more accurately represent the area after synthesis. Since the benchmarks running on the reconfigurable architecture all use the same architecture description, the area is the same for all benchmarks.

The area of Blocks is 1.41 square millimeters and the model estimates an area of 1.38 square millimeters. Therefore, the mean absolute percentage error for the reconfigurable architecture instance is 2.1%. Figure 17 shows the area breakdown for the reconfigurable architecture. For the individual categories, the MAPEs are: 8.4%, 1.3%, 16.1%, 2.4%, and 14.3% for function units, global memory, instruction memories, local memories, and switch-boxes, respectively. The instruction memory sizes and area attributed to the reconfigurable network therefore have the largest estimation error. Similar to the hard-wired architecture instances, the area for the instruction memories is overestimated. There is also overestimation with regard to the switch-box area. This is because for the estimation of the area of a network a switch-box is used that has connections on all four sides (top, right, bottom, and left) as well as connections to the outputs and inputs of a function unit.

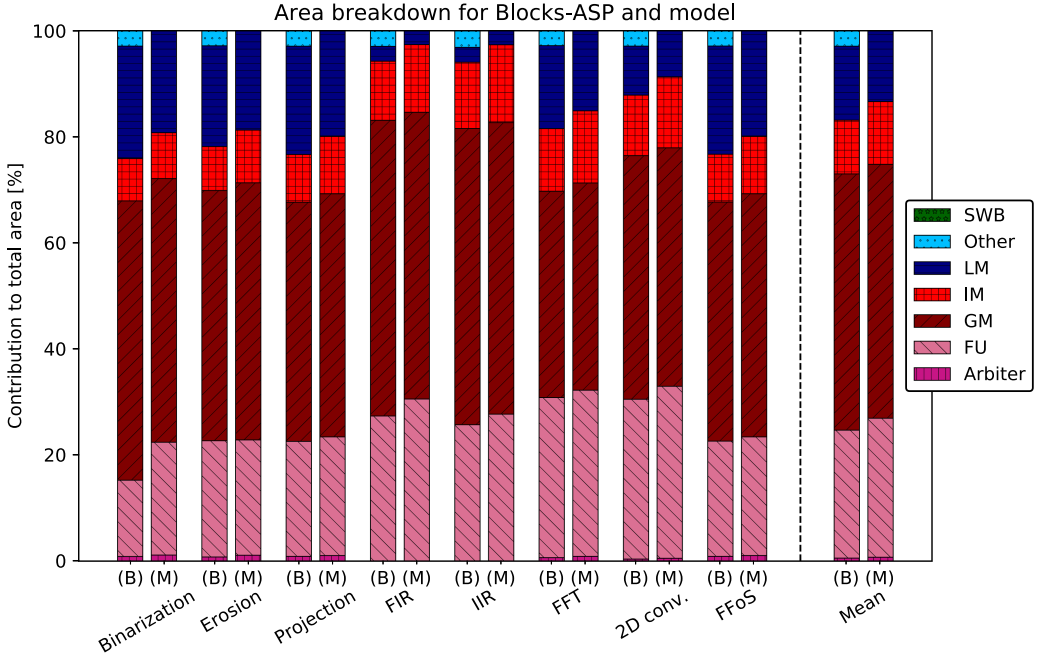


Fig. 16. Area breakdown for the hard-wired architectures, according to the synthesis reports (B) and the model (M).

This is not the case for all switch-boxes in the network, for example a switch-box in the upper left corner of the grid structure will only have connections on the right and bottom as well as possibly connections to function unit inputs. This is something that could be improved upon the model, at the expense of some extra complexity. The reason for the slight underestimation, for the function units, can most likely be found in scaling of gates during synthesis to meet timing requirements. This could be due to synthesis constraints. For example, the capacity on the outputs of a function unit can be different for the whole design compared to for a single synthesized unit. Alternatively, it can be that constraints on combinational paths are different, leading to different synthesis results. These differences are almost unavoidable when components are individually synthesized.

5.4 Model Calculation Time

Since the execution time required by the model is important when performing many DSE iterations, an analysis on the running time has been performed. Figure 18 shows model execution times for both the hard-wired (Model-ASP) and reconfigurable (Model) architectures. In both cases, the computation time for estimating the energy and area results correlate with the number of cycles it takes to execute the benchmark application. This is caused by the fact that the model analyzes an execution trace to obtain information for energy estimation.

The execution time for the model for a reconfigurable architecture takes longer to complete. This is mainly caused by estimating the energy for the switch-boxes. To do so, the place-and-route file specifying configured paths through the switch-boxes is analyzed to extract the paths between function units. This calculation could also be performed when the architecture configuration is generated, moving this time away from the model and allowing some speedup.

When compared to a full synthesis run, the model provides an average speedup of close to three orders of magnitude. However, it is possible to make optimizations to the synthesis flow in

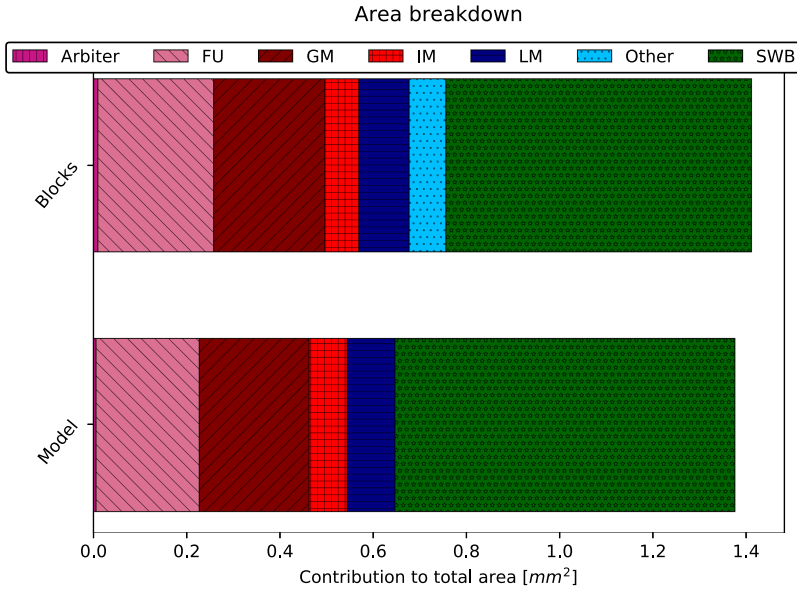


Fig. 17. Area breakdown of the synthesis tools (Blocks) and the model for the reconfigurable architecture.

some cases. For example, the design only has to be simulated if the instruction schedule changes. However, when function units are added, various aspects of the design change: the network, the arbiter, and so on. In this case, synthesis is required. However, it is possible to shorten the synthesis time by using pre-synthesized IP blocks for individual function components. After this synthesis, the design has to be functionally simulated. However, in both cases, the model will still provide a significant speedup.

Additionally, the model does not require access to ASIC technology libraries, which are often sensitive intellectual property. The energy and area model allows implementation of DSE tools for Blocks that provide fast energy, area, and performance tradeoffs.

6 CONCLUSIONS AND FUTURE WORK

Reconfigurable architectures provide many options for configuration and application mapping. Finding a good implementation solution is, therefore, not a trivial task. Automated DSE can help a designer to find a good application mapping solution. In the case of CGRAs, DSE tools require both energy and area estimations to estimate whether a design will fit on a given amount of chip area and if it is more energy-efficient than another design. It is important that such estimations are both accurate and fast. Speed is important, as a reduced estimation time will allow for more design iterations, possibly leading to better solutions.

The proposed energy model is calibrated on individual function units. These function units were stimulated on post-synthesis net-lists for all available instructions and a random sequence of input data with a predetermined data-width. As such, energy numbers are known for every data type supported by Blocks. This is a one-time effort. The numbers are used by the energy model, in combination with an instruction trace, to estimate the energy required to execute a given application. The instruction trace can be obtained by simulation or by a compiler. The results show that the energy model has a mean absolute percentage error error of 10.7% for hard-wired and 15.5% for reconfigurable Blocks instances. The energy model demonstrates a maximum error of 39% and 29% for hardwired and reconfigurable instances, respectively, on the evaluated benchmark set.

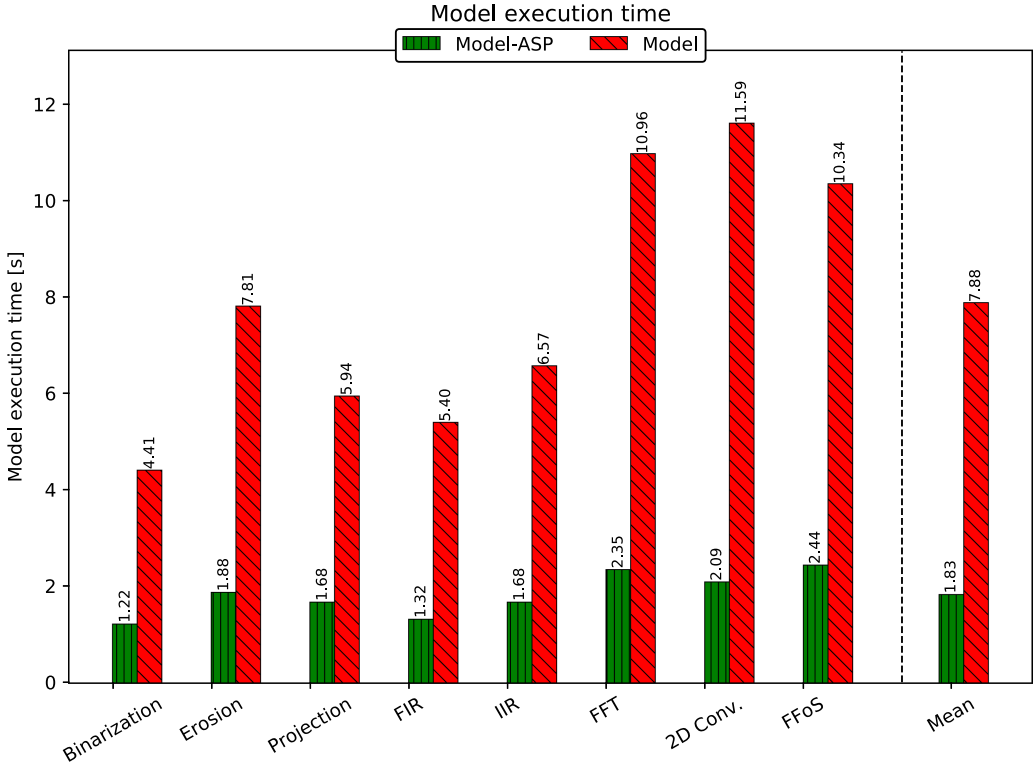


Fig. 18. Execution time of the model for the evaluated benchmarks. All measurements are performed with a single threaded application on an Intel Xeon E5-2620 v2.

Area is estimated by analyzing the architecture description. The corresponding area for each of the building blocks in the design is looked up from a database of post-synthesis results. Summed up, the individual area numbers of subsystems provide the total design area. The area model has an error margin between -4% and $+4\%$ for hard-wired Blocks instances and -2.1% for reconfigurable instances. The mean absolute percentage errors for hardwired and reconfigurable areas are 2.5% and 2.1% , respectively.

Since the energy and area models do not require a full synthesis run of a design, the estimation time is significantly reduced. For reconfigurable architectures, the model execution time is less than eight seconds, whereas a synthesis run takes over two hours to complete. Thus, the total execution time improvement is close to three orders of magnitude.

6.1 Future Work

The model can be made more accurate by several improvements, as outlined in the evaluation section. Although these may result in a slower execution time of the model, it might be worth implementing these and evaluate their impact on the decisions of a design space exploration tool. The development of a design space exploration tool that chooses a hardware configuration is currently in progress.

Another possibility, which is currently supported by the building block characterization, is to obtain an energy distribution for each operation. This makes it possible to compute with probability distributions instead of the median energy consumption. The outcome of such a computation is another distribution, but for the entire design. Typically this cannot be used directly by a cost

function (the median of the distribution could be used, but this would give the same result as the current median). However, it may be interesting to design a DSE tool that operates on cost probability distributions.

REFERENCES

- [1] D. Chen, J. Cong, Y. Fan, and Z. Zhang. 2007. High-level power estimation and low-power design space exploration for FPGAs. In *Asia and South Pacific Design Automation Conference*. IEEE, New York, NY, 529–534. DOI: <https://doi.org/10.1109/ASPDAC.2007.358040>
- [2] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, and J. Anderson. 2017. CGRA-ME: A unified framework for CGRA modelling and exploration. In *IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP'17)*. IEEE, New York, NY, 184–189. DOI: <https://doi.org/10.1109/ASAP.2017.7995277>
- [3] Lanping Deng, K. Sobti, and C. Chakrabarti. 2008. Accurate models for estimating area and power of FPGA implementations. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP'08)*. IEEE, New York, NY.
- [4] Nagu Dhanwada, David Hathaway, Victor Zyuban, Peng Peng, Karl Moody, William Dungan, Arun Joseph, Rahul Rao, and Christopher Gonzalez. 2013. Efficient PVT independent abstraction of large IP blocks for hierarchical power analysis. In *IEEE/ACM International Conference on Computer-aided Design (ICCAD'13)*. IEEE, New York, NY, 458–465.
- [5] Tobias Grosser, Armin Groesslinger, and Christian Lengauer. 2012. Polly—Performing Polyhedral Optimizations on a Low-level Intermediate Representation. 1250010 pages.
- [6] Reiner Hartenstein. 2001. A decade of reconfigurable computing: A visionary retrospective. In *Conference on Design, Automation and Test in Europe*. IEEE, New York, NY.
- [7] Z. Hou, Z. Zhao, W. Sheng, and W. He. 2016. System level power consumption modeling and optimization for coarse-grained reconfigurable architectures. In *International Conference on Integrated Circuits and Microsystems (ICIM'16)*. IEEE, New York, NY, 1–6. DOI: <https://doi.org/10.1109/ICAM.2016.7813532>
- [8] Y. Kim, R. N. Mahapatra, and K. Choi. 2010. Design space exploration for efficient resource utilization in coarse-grained reconfigurable architecture. *IEEE Trans. Very Large Scale Integ. (VLSI) Syst.* 18, 10 (2010), 1471–1482. DOI: <https://doi.org/10.1109/TVLSI.2009.2025280>
- [9] R. Koenig, L. Bauer, T. Stripf, M. Shafique, W. Ahmed, J. Becker, and J. Henkel. 2010. KAHRISMA: A novel hypermorphic reconfigurable-instruction-set multi-grained-array architecture. In *Design, Automation Test in Europe Conference Exhibition (DATE'10)*. IEEE, New York, NY, 819–824. DOI: <https://doi.org/10.1109/DATE.2010.5456939>
- [10] Akash Kumar, Shakith Fernando, Yajun Ha, Bart Mesman, and Henk Corporaal. 2008. Multiprocessor systems synthesis for multiple use-cases of multiple applications on FPGA. *ACM Trans. Des. Autom. Electron. Syst.* 13, 3 (July 2008). DOI: <https://doi.org/10.1145/1367045.1367049>
- [11] Fei Li, Deming Chen, Lei He, and Jason Cong. 2003. Architecture evaluation for power-efficient FPGAs. In *ACM/SIGDA 11th International Symposium on Field-programmable Gate Arrays (FPGA'03)*. ACM, New York, NY.
- [12] Cheng Liu and Hayden Kwok-Hay So. 2015. Automatic soft CGRA overlay customization for high-productivity nested loop acceleration on FPGAs. In *IEEE 23rd International Symposium on Field-programmable Custom Computing Machines*. IEEE, New York, NY, 101–101.
- [13] Dajiang Liu, Shouyi Yin, Leibo Liu, and Shaojun Wei. 2013. Polyhedral model based mapping optimization of loop nests for CGRAs. In *50th Design Automation Conference*. AMC, New York, NY, 1–8.
- [14] A. S. B. Lopes and M. M. Pereira. 2020. A machine learning approach to accelerating DSE of reconfigurable accelerator systems. In *33rd Symposium on Integrated Circuits and Systems Design (SBCCI'20)*. IEEE, New York, NY, 1–6. DOI: <https://doi.org/10.1109/SBCCI50935.2020.9189899>
- [15] Bingfeng Mei, Serge Vernalde, Diederik Verkest, Hugo De Man, and Rudy Lauwereins. 2003. ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix. In *Field Programmable Logic and Application*. Springer, Berlin, Germany.
- [16] S. Nalesh, K. T. Madhu, S. Das, S. K. Nandy, and R. Narayan. 2015. Energy aware synthesis of application kernels expressed in functional languages on a coarse grained composable reconfigurable array. In *IEEE International Symposium on Nanoelectronic and Information Systems*. IEEE, New York, NY, 7–12. DOI: <https://doi.org/10.1109/INIS.2015.39>
- [17] Alejandro Nocua, Arnaud Virazel, Alberto Bosio, Patrick Girard, and Cyril Chevalier. 2016. A hybrid power estimation technique to improve IP power models quality. In *IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-Soc'16)*. IEEE, New York, NY, 1–6.
- [18] Roel Pieters, Pieter Jonker, and Henk Nijmeijer. 2009. *Real-time Center Detection of an OLED Structure*. Springer Berlin, 400–409. DOI: https://doi.org/10.1007/978-3-642-04697-1_37

- [19] Karthikeyan Sankaralingam, Ramadass Nagarajan, Haiming Liu, Changkyu Kim, Jaehyuk Huh, Nitya Ranganathan, Doug Burger, Stephen W. Keckler, Robert G. McDonald, and Charles R. Moore. 2004. TRIPS: A polymorphous architecture for exploiting ILP, TLP, and DLP. *ACM Trans. Archit. Code Optim.* 1 (2004), 62–93.
- [20] Shyamkumar Thoziyoor, Naveen Muralimanohar, Jung Ho Ahn, and Norman P. Jouppi. 2008. *CACTI 5.1*. Technical Report. Technical Report HPL-2008-20, HP Labs.
- [21] E. Waingold, M. Taylor, D. Srikrishna, V. Sarkar, W. Lee, V. Lee, J. Kim, M. Frank, P. Finch, R. Barua, J. Babb, S. Amarasinghe, and A. Agarwal. 1997. Baring it all to software: Raw machines. *Computer* 30, 9 (Sep. 1997), 86–93. DOI : <https://doi.org/10.1109/2.612254>
- [22] M. Wijnvliet, L. Waeijen, and H. Corporaal. 2016. Coarse grained reconfigurable architectures in the past 25 years: Overview and classification. In *International Conference on Embedded Computer Systems: Architectures, Modeling and Simulation (SAMOS'16)*. IEEE, New York, NY, 235–244. DOI : <https://doi.org/10.1109/SAMOS.2016.7818353>
- [23] M. Wijnvliet, L. Waeijen, and H. Corporaal. 2019. Blocks: Redesigning coarse grained reconfigurable architectures for energy efficiency. In *29th International Conference on Field-programmable Logic and Applications (FPL'19)*. IEEE, New York, NY, 235–244.
- [24] Dennis Leander Wolf, Christoph Spang, and Christian Hochberger. 2020. Towards purposeful design space exploration of heterogeneous CGRAs: Clock frequency estimation. In *57th ACM/IEEE Design Automation Conference (DAC'20)*. IEEE, New York, NY, 1–6.
- [25] Zhongyuan Zhao, Yantao Liu, Weiguang Sheng, Tushar Krishna, Qin Wang, and Zhigang Mao. 2018. Optimizing the data placement and transformation for multi-bank CGRA computing system. In *Design, Automation & Test in Europe Conference & Exhibition (DATE'18)*. IEEE, New York, NY, 1087–1092.

Received December 2020; revised May 2021; accepted May 2021